FixIt Local — Full Project Documentation

Graduation Project — DEPI Program A Flutter mobile application that connects homeowners with trusted local service professionals.

Table of Contents

1. Project Overview

FixIt Local solves a real problem: finding a reliable home-service professional in your area is usually a frustrating experience. You either rely on word-of-mouth, or end up scrolling through generic websites with no sense of who's actually nearby.

This app fixes that. It gives homeowners a clean, intuitive way to browse local experts — plumbers, electricians, HVAC technicians, cleaners — see their ratings, distance, and pricing upfront, and eventually book them directly. On the flip side, service providers get dedicated tools to build their profile, showcase work photo portfolios, manage daily job schedules, and toggle online job availability.

The app is built entirely in Flutter and uses Firebase Firestore as its backend, making it ready to scale without managing any server infrastructure.

Key capabilities:

- Browse nearby professionals, sorted by distance
- Real-time keyword search and filtering by category, rating, and proximity
- View detailed provider profiles with work photo portfolios and offered services
- Service providers can build and complete their own public profiles
- Service providers can manage their daily booking schedule with an online/offline job acceptance switch
- Complete payment method management: saved credit/debit cards, express checkout (Apple Pay/Google Pay), secure SSL encryption banner, and transaction history logs
- Shimmer loading states for a polished, responsive feel
- Supports both light and dark themes

2. Screenshots & UI Walkthrough

The following sections describe the screen flows and UI architecture of the application.

Authentication Flow

The app opens with a **splash screen** showing the FixIt Local logo and tagline. From there, users land on the **Sign Up** / **Login** flow. The design intentionally keeps things minimal — a clean white card, the FixIt Local

branding at the top, and a single call-to-action button. There's also a social login shortcut (Google, Facebook) to lower friction for new users.

The **Forgot Password** flow is three steps: enter your email, receive a 6-digit OTP, enter it on the verification screen, then reset your password. The OTP input uses a pin-field widget (Pinput package) for a native, digit-by-digit experience.

Home Screen

Once logged in, users see the **Home** screen inside a bottom navigation shell. It shows:

- A headline search bar ("What needs fixing today?") that navigates to the full Search screen when tapped
- A horizontal row of **Popular Service** category icons (Plumber, Electrician, Cleaner, HVAC Tech) — tapping any of these jumps straight to Search pre-filtered by that category
- An **emergency CTA banner** (dark background, "Emergency help needed? Find pros available within 1 hour")
- A **Nearby Professionals** list, loaded from Firestore and displayed as cards with avatar, name, specialty, distance, rating badge, and hourly rate

Search Screen

The search screen gives users full control to narrow down results:

- Real-time text search across provider name, specialty, and category
- Quick-filter chips (Plumbing, Electrical, Cleaning, HVAC)
- An expandable **Filter Panel** with toggle switches for "Nearby" (≤ 1.5 miles) and "Top Rated" (≥ 4.8 stars)

- Results are sorted by distance when Nearby is active
- Friendly "Ready to find your pro?" empty state when no results match

Provider Profile (Complete Profile)

This screen is for service providers setting up their account. It's divided into four sections stacked in a scrollable column:

1. **Profile Photo** — A circular avatar picker with a camera overlay button. Tapping it opens the device camera.
2. **Personal Details** — Full name, primary specialty, years of experience, and an "About Me" multi-line text field.
3. **Services Offered** — Animated chips for each selected service with an "×" to remove them. A "Add Specialty" button opens a draggable bottom sheet with the full catalog of available services.
4. **Photos of Your Work** — A 2-column grid where providers can upload up to 10 portfolio photos from camera or gallery. Each photo tile has a delete button overlay.

Booking & Provider Detail Views

From the design mockups you can see several additional screens that are part of the complete user journey:

- **Provider Detail Page** — Shows the full provider card with their reliability score (98%), service coverage area, next availability, photos of work, and customer reviews.
- **Booking Slot Picker** — A calendar-style date/time picker to choose an appointment slot.
- **Payment Methods** — Saved cards, Apple Pay, Google Pay, security encryption banner, and transaction history.

- **Appointment Confirmed** — A success screen summarizing who you booked, when, for what service, and where.
- **Provider Dashboard ("Manage Your Day")** — The provider's home view showing today's upcoming bookings, online availability toggle switch, "NEXT UP" job card with navigation and message shortcuts.

[!NOTE] The Provider Dashboard and Payment Methods screens are implemented and integrated into the codebase. Home, Search, Provider Profile, Provider Dashboard, and Payment Methods screens are fully functional.

3. Tech Stack & Dependencies

Dependency	Version	Why it's here
Flutter SDK	^3.10.4	Core framework
`firebase_core`	^4.11.0	Firebase initialization
`cloud_firestore`	^6.7.1	Cloud database for providers and customers
`go_router`	^17.1.0	Declarative routing with URL parameters
`flutter_bloc`	^9.1.1	State management (BLoC pattern)
`cached_network_image`	^3.4.1	Loads and caches network images efficiently
`image_picker`	^1.2.2	Camera and gallery access for profile/work photos
`shimmer`	^3.0.0	Skeleton loading animations

`shared_preferences`	^2.5.4	Lightweight local key-value storage
`hive_ce` + `hive_ce_flutter`	^2.19.3 / ^2.3.4	Fast offline-capable local database
`gap`	^3.0.1	Adds clean spacing between widgets
`flutter_svg`	^2.2.3	SVG icon rendering
`lottie`	^3.3.2	After Effects / Lottie animations
`pinput`	^6.0.2	OTP/PIN input field
`percent_indicator`	^4.2.5	Circular and linear progress indicators
`flutter_rating_bar`	^4.0.1	Star rating display
`carousel_slider`	^5.1.2	Image carousels
`date_picker_timeline`	^1.2.7	Horizontal date timeline picker
`intl`	^0.20.2	Date/number formatting and localization
`readmore`	^3.0.0	Expandable "Read more" text
`smooth_page_indicator`	^2.0.1	Dot indicators for page views
`flutter_slidable`	^4.0.3	Swipeable list items
`dio`	^5.9.2	HTTP client for REST API calls
`chili_debug_view`	^1.3.1	In-app debug panel overlay

Dev dependencies:

- `build_runner` — Code generation (Hive adapters)

- `hive_ce_generator` — Generates Hive type adapters
- `flutter_lints` — Lint rules

4. Project Architecture

The project follows a **Feature-First Clean Architecture** pattern. Each feature is self-contained under `lib/features/`, and shared code lives in `lib/core/`.

Within each feature the layers are:

- **`presentation/page/`** — The actual screen widget (the thing you navigate to)
- **`presentation/widgets/`** — Smaller, composable UI pieces used by that page

This separation keeps screens from becoming monolithic. When a screen gets complex, you extract part of it into a widget file rather than piling more build methods into a single class.

The `core/` layer is strictly shared infrastructure — things that multiple features depend on. Nothing in `core/` should import from `features/`.

```
Architecture:
  Feature → Presentation (Page + Widgets)
  Feature → Core (Models, Services, Styles, Widgets, Rout
  Core ← Firebase / SharedPrefs / SharedPreferences
```

State management is handled with **Flutter BLoC** where needed. For simpler cases like search filters or dashboard toggles, plain `setState` inside `StatefulWidget` is used.

5. Directory Structure

```
fixit-local/
├── lib/
│   ├── main.dart                    # Entry point & mock
│   ├── main_app.dart                # Root MaterialApp.r
│   ├── firebase_options.dart        # Auto-generated Fir
│   │
│   ├── core/
│   │   ├── constants/
│   │   │   ├── app_assets.dart      # All asset path con
│   │   │   └── app_fonts.dart       # Font family name c
│   │   ├── functions/
│   │   │   ├── navigations.dart     # push/pop/go_router
│   │   │   ├── custom_snake_bar.dart# SnackBar helper
│   │   │   ├── upload.dart          # Upload helper stub
│   │   │   └── validation.dart      # Input validation f
│   │   ├── models/
│   │   │   ├── user_model.dart      # Base user data mod
│   │   │   ├── customer_model.dart  # Extends UserModel
│   │   │   └── provider_model.dart  # Extends UserModel
│   │   ├── routes/
│   │   │   ├── app_router.dart      # GoRouter config &
│   │   │   └── routes.dart          # Route path constan
│   │   ├── services/
│   │   │   ├── firebase_service.dart# Firestore CRUD ope
│   │   │   └── local/
│   │   │       └── shared_pref.dart # SharedPreferences
│   │   ├── shimmer/
│   │   │   ├── shimmer_widget.dart  # Base shimmer wrapp
│   │   │   ├── list_shimmer.dart    # List skeleton load
│   │   │   ├── grid_shimmer.dart    # Grid skeleton load
│   │   │   └── book_card_shimmer.dart # Booking placehol
│   │   ├── styles/
│   │   │   ├── app_colors.dart      # Color palette cons
│   │   │   ├── text_styles.dart     # Text style definit
│   │   │   └── themes.dart          # Light + dark Theme
│   │   └── widgets/
│   │       ├── custom_form_field.dart    # Reusable text
│   │       ├── custom_image_picker.dart  # Avatar image
│   │       ├── custom_password_field.dart# Password inpu
│   │       ├── filled_icon_button.dart   # Circular icon
```

```
|   |       ├── image_container.dart      # Rounded image
|   |       ├── main_button.dart          # Primary CTA b
|   |       ├── mybodyview.dart           # Consistent sc
|   |       └── svg_pic.dart              # SVG rendering
|   |
|   └── features/
|       ├── main_home/
|       |   └── presentation/
|       |       └── main_home.dart        # Bottom naviga
|       ├── home/
|       |   └── presentation/
|       |       ├── page/home.dart        # Home screen v
|       |       └── widgets/
|       |           ├── icons_builder.dart     # Category
|       |           ├── message.dart           # Emergenc
|       |           ├── mock_professionals.dart # Mock da
|       |           └── professional_card.dart  # Provide
|       ├── search/
|       |   └── presentation/
|       |       ├── page/search_screen.dart    # Search s
|       |       └── widgets/
|       |           ├── search_input_field.dart# Search f
|       |           ├── location_row.dart      # Location
|       |           ├── filter_panel.dart       # Filter
|       |           └── build_results.dart     # Results
|       ├── provider_info/
|       |   └── presentation/
|       |       ├── page/provider_info.dart    # Complete
|       |       └── widgets/
|       |           ├── custom_container.dart       # Sec
|       |           ├── services_offered_section.dart # S
|       |           ├── photos_of_work_section.dart   # P
|       |           ├── __photo_tile.dart             # P
|       |           └── __upload_tile.dart            # U
|       ├── provider_dashboard/
|       |   └── presentation/
|       |       └── page/
|       |           └── provider_dashboard_page.dart # Da
|       └── payment/
|           └── presentation/
```

```
│                └── page/
│                    └── payment_page.dart # Payment metho
│
├── assets/
│   ├── images/        # PNG/JPG images (user_placeholder.
│   ├── icons/         # SVG icons (home, book, profile, n
│   ├── lottie/        # Lottie JSON animation files
│   └── fonts/         # Manrope font family (Bold, SemiBo
│
├── pubspec.yaml
└── firebase.json
```

6. App Entry Point

File: `main.dart`

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  // Seed mock data for development
  await AppFirebaseService.sendData();
  runApp(const MainApp());
}
```

Three things happen on startup:

1. **`WidgetsFlutterBinding.ensureInitialized()`** — Required before any async work before `runApp`. Makes sure the Flutter engine is ready.

2. **`Firebase.initializeApp()`** — Bootstraps Firebase using the platform-specific options generated by `flutterfire configure`.

3. **`AppFirebaseService.sendData()`** — Seeds the Firestore

database with dummy providers and a default customer if the collections are empty. This is a development convenience so you don't need to manually populate Firestore every time.

File: `main_app.dart`

```
class MainApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp.router(
      routerConfig: AppRouter.routes,
      debugShowCheckedModeBanner: false,
      theme: AppThemes.lightTheme,
      darkTheme: AppThemes.darktheme,
      themeMode: ThemeMode.light,
      builder: (context, child) {
        return DebugView(
          app: child,
          navigatorKey: navigatorKey,
          showDebugViewButton: true,
        );
      },
    );
  }
}
```

`MaterialApp.router` is used because the app uses `GoRouter` for navigation. The `DebugView` wrapper from `chili_debug_view` adds an in-app overlay button that exposes network logs and debug info during development.

7. Routing

Files:

- `app_router.dart`
- `routes.dart`

Route paths are kept as string constants in `Routes` to avoid typos when navigating:

```
class Routes {
  static const String splash            = '/';
  static const String welcome           = '/welcome';
  static const String login             = '/login';
  static const String register          = '/register';
  static const String home              = '/home';
  static const String search            = '/search';
  static const String providerInfo      = '/providerInfo'
  static const String providerDashboard = '/providerDashb
}
```

The router is a `GoRouter` instance configured in `AppRouter`:

```
class AppRouter {
  static final routes = GoRouter(
    navigatorKey: navigatorKey,
    routes: [
      GoRoute(
        path: Routes.providerInfo,
        builder: (context, state) => const ProviderInfo()
      ),
      GoRoute(
        path: Routes.splash,
        builder: (context, state) => const MainHome(),
      ),
      GoRoute(
        path: Routes.search,
        builder: (context, state) {
          final category = state.uri.queryParameters['cat
          return SearchScreen(initialCategory: category);
        },
      ),
      GoRoute(
        path: Routes.providerDashboard,
        builder: (context, state) => const ProviderDashbo
      ),
```

```
      ],
    );
  }
```

Notice how the Search route reads a `category` query parameter from the URL. This is how the category icons on the Home screen pre-filter the search: they push `/search?category=Plumbing`, and `SearchScreen` picks that up via `state.uri.queryParameters['category']`.

The `navigatorKey` is a `GlobalKey<NavigatorState>` declared at the top level so the `DebugView` can hook into the same navigator.

8. Data Models

UserModel

File: `user_model.dart`

The base class for any user in the system. Both customers and providers extend this.

```
class UserModel {
  final String uId;
  final String name;
  final String email;
  final String phone;
  final String role;    // 'customer' or 'provider'
  final String? imageUrl;

  UserModel({
    required this.uId,
    required this.name,
    required this.email,
    required this.phone,
    required this.role,
```

```
    this.imageUrl,
  });

  factory UserModel.fromJson(Map<String, dynamic> json) {
  Map<String, dynamic> toJson() { ... }
}
```

The `role` field is the discriminator. `fromJson` handles both `uId` and `uid` key variants for Firestore compatibility.

CustomerModel

File: `customer_model.dart`

Extends `UserModel` and adds an `address` field.

```
class CustomerModel extends UserModel {
  final String? address;

  CustomerModel({
    required super.uId,
    required super.name,
    required super.email,
    required super.phone,
    required super.role,
    super.imageUrl,
    this.address,
  });

  @override
  Map<String, dynamic> toJson() {
    final data = super.toJson();
    data['address'] = address;
    return data;
  }
}
```

ProviderModel

File: `provider_model.dart`

The most data-rich model in the app. Extends `UserModel` and adds everything needed to display a professional's card and profile.

```
class ProviderModel extends UserModel {
  final double rating;
  final String specialty;
  final String category;
  final double distance;
  final double pricePerHour;
  final bool isTopRated;
  final String? about;
  final List<String>? servicesOffered;

  ProviderModel({ ... });
}
```

The `fromJson` factory includes a local `parseDouble` helper to safely coerce Firestore's dynamic number types (`int`, `double`, or `String`) into a Dart `double`.

```
double parseDouble(dynamic val) {
  if (val == null) return 0.0;
  if (val is num) return val.toDouble();
  return double.tryParse(val.toString()) ?? 0.0;
}
```

BookingModel

File: `provider_dashboard_page.dart`

Model representing a scheduled booking appointment on the provider dashboard:

```
class BookingModel {
  final String name;
  final String service;
  final String time;
  final String date;
  final String distance;
  final bool isNext;

  BookingModel({
    required this.name,
    required this.service,
    required this.time,
    required this.date,
    required this.distance,
    required this.isNext,
  });
}
```

CardModel

File: `payment_page.dart`

Model representing a user's saved credit or debit payment card:

```
class CardModel {
  final String type;
  final String lastFour;
  final String expiry;
  final bool isDefault;

  CardModel({
    required this.type,
    required this.lastFour,
    required this.expiry,
    required this.isDefault,
  });
}
```

TransactionModel

File: `payment_page.dart`

Model representing a completed billing line item / transaction record:

```
class TransactionModel {
  final String title;
  final String date;
  final String amount;

  TransactionModel({
    required this.title,
    required this.date,
    required this.amount,
  });
}
```

9. Services Layer

AppFirebaseService

File: `firebase_service.dart`

All Firestore interactions go through this static service class.

```
class AppFirebaseService {
  static final fireStore = FirebaseFirestore.instance;
  static final providers = fireStore.collection('provider
  static final customers = fireStore.collection('customer
}
```

Methods:

Method	What it does
`createCustomer(CustomerModel)`	Writes a customer doc to Firestore, keyed by `uId`
`createProvider(ProviderModel)`	Writes a provider doc to Firestore, keyed by `uId`
`getProvider(String id)`	Fetches a single provider document
`getAllProviders()`	Returns all providers ordered by rating descending
`getNearbyProviders()`	Returns all providers ordered by distance ascending
`sendData()`	Dev seeder — populates Firestore with mock providers and customer if empty

Mock providers seeded:

1. **QuickDrain Plumbing** — 4.7★, Emergency Repairs, $80/hr, 0.8 mi
2. **Sparky Lights Electrical** — 4.9★, Wiring & Lighting, $95/hr, 1.2 mi
3. **EcoCool HVAC Services** — 4.6★, AC & Heating, $110/hr, 2.5 mi
4. **ShineBright Cleaning** — 4.8★, Deep House Cleaning, $45/hr, 1.5 mi

SharedPref

File: `shared_pref.dart`

A static wrapper around the `shared_preferences` package. All keys are defined as constants at the top of the class:

```
class SharedPref {
  static const ktoken     = "token";
  static const kuser      = "user";
  static const kotp       = "otp";
  static const kuserId    = "forgot_user_id";
  static const kfavourite = "favourite";
}
```

Provides typed getters/setters for auth token, OTP, user ID, favourites list, and generic key-value pairs.

10. Features — Screen by Screen

10.1 Main Shell — Bottom Navigation

File: `main_home.dart`

`MainHome` is the app's shell. It's a `StatefulWidget` that holds the current tab index and renders the right screen in its body. The three tabs are Home, Bookings, and Profile.

The `AppBar` shows the app title ("FixIt Local") centered, a hamburger menu on the left, and a notification bell (SVG icon) on the right.

The `NavigationBar` (Material 3 style) uses custom SVG icons from `AppAssets`. Selected icon tints to `AppColors.primaryColor` (navy blue).

10.2 Home Screen

File: `home.dart`

A `StatelessWidget` (data fetching happens via `FutureBuilder`).

Layout (top to bottom):

1. **"Find your local expert"** headline — `TextStyles.headline` (28px, Manrope Medium)
2. Search field (`CustomFormField`) in `readOnly: true` mode. Tapping it pushes `/search`.
3. **"Popular Services" row** — label on left, "View all" link on right. `IconsBuilder` renders horizontal scrolling category chips.
4. **Emergency banner** (`Message` widget) — dark card with warning icon, text, and "View Pros" button.
5. **"Nearby Professionals"** section — `FutureBuilder` calling `AppFirebaseService.getNearbyProviders()`. Shows `ListShimmer(itemCount: 3)` while loading, then renders `ProfessionalCard` list.

Widget: `IconsBuilder`

File: `icons_builder.dart`

Renders a horizontal row of category icons (Plumbing, Electrical, Cleaning, HVAC). Tapping pushes `${Routes.search}?category=${category['name']}`.

Widget: `Message`

File: `message.dart`

Emergency CTA banner container (`AppColors.black1` #131B2E) with warning SVG and "View Pros" button.

Widget: `ProfessionalCard`

File: `professional_card.dart`

Renders provider list cards with avatar image (`CachedNetworkImage` or local asset), rating badge, specialty, distance, starting rate ($/hr), and chevron navigation icon.

10.3 Search Screen

File: `search_screen.dart`

`StatefulWidget` managing search state (`_searchController`, `_showFilters`, `_selectedQuickFilter`, `_isNearby`, `_isTopRated`).

- Client-side filtering logic filters by query text, selected category chip, top-rated toggle ($\ge 4.8\star$), and nearby toggle (≤ 1.5 mi).
- Uses `AnimatedSwitcher` to transition between the filter panel and the results list.

Widget: **`SearchInputField`**

File: `search_input_field.dart`

Wrapper around `CustomFormField` with search icon prefix and filter SVG suffix button.

Widget: **`FilterPanel`**

File: `filter_panel.dart`

Filter configuration view with horizontally scrollable quick filter chips, `Switch.adaptive` essential filter switches, and an Apply Filters button.

10.4 Provider Profile / Complete Profile

File: `provider_info.dart`

Profile setup screen for service providers:

1. **Profile Photo Card** → `CustomImagePicker` avatar selector.
2. **Personal Details Card** → `CustomFormField` inputs for name, specialty, experience, and about text.
3. **Services Offered Card** → `ServicesOfferedSection` displaying selected service chips with an "Add Specialty" draggable bottom sheet.

4. **Photos of Work Card** → `PhotosOfWorkSection` 2-column portfolio photo grid with camera/gallery picker and delete overlay buttons.

10.5 Provider Dashboard ("Manage Your Day")

File: `provider_dashboard_page.dart`

Dedicated home screen for service providers to manage daily work:

- **Accepting Jobs Switch**: `StatefulWidget` managing the `isAcceptingJobs` boolean switch. When toggled off (Offline mode), the provider is hidden from customer search.
- **Bookings Counter & Header**: Displays headline "Manage Your Day" and total count of scheduled bookings for today.
- **Upcoming Bookings Queue**: `FutureBuilder` calling `fetchBookings()` to retrieve scheduled `BookingModel` items.
- **"NEXT UP" Highlight Card**: Formatted card for the immediate upcoming job showing client avatar, name, distance ("2.4 miles away"), appointment time ("09:30 AM"), and date ("Today, Oct 24"). Includes primary action buttons:
  - **Start Navigation**: `ElevatedButton` with `AppColors.titlecolor` fill.
  - **Message**: `OutlinedButton` for client messaging.
- **Scheduled Appointments List**: Cards displaying client avatar, name, service category, time slot, and chevron icon for detailed view.

10.6 Payment Methods & Billing

File: `payment_page.dart`

Comprehensive payment management view for users and providers:

- **Saved Cards Section**: Powered by `fetchPaymentData()`, rendering a list of saved `CardModel` payment cards (`_CardTile`):
  - Brand credit card icon (Visa in `AppColors.titlecolor`,

Mastercard in `AppColors.redcolor`).
    - Masked last 4 digits (e.g., Visa ending in 4242).
    - Expiration date and **Default** status badge indicator.
    - Action buttons for editing and deleting saved cards.
    - `DottedAddCardButton`: Bordered container button triggering card addition.
- **Express Checkout**: Grid of `_ExpressCheckoutTile` widgets showcasing digital wallet integration status:
    - **Apple Pay**: Status label "Enabled" (green badge).
    - **Google Pay**: Status label "Setup" (grey badge).
- **Secure Payments Banner**: Highlighted container (`AppColors.titlecolor`) with shield icon explaining data encryption and server storage compliance.
- **Recent Transactions & Billing History**: Itemized list of recent `TransactionModel` payments (title, date, formatted amount) and a button to view full billing history.

11. Core Shared Widgets

`CustomFormField`

File: `custom_form_field.dart`

Standard text input field wrapping `TextFormField` with `InputDecorationTheme` styling.

`CustomImagePicker`

File: `custom_image_picker.dart`

Circular avatar picker with camera icon badge overlay in the bottom right, integrated with `image_picker`.

`MyBodyView`

File: `mybodyview.dart`

`Padding` widget applying consistent horizontal margin padding across all screen bodies.

`SvgPic`

File: `svg_pic.dart`

Wrapper around `flutter_svg`'s `SvgPicture.asset`.

12. Design System

12.1 Color Palette

File: `app_colors.dart`

Token	Hex	Usage
`primaryColor`	`#00174B`	Primary actions, selected icons, borders
`titlecolor`	`#0051D5`	Blue links, titles, active elements
`bodycolor`	`#677294`	Body/secondary text
`navicon`	`#858EA9`	Navigation bar icons
`secondaryColor`	`#F5EFE1`	Warm cream backgrounds
`blackColor`	`#191C1E`	Primary text
`redcolor`	`#F51212`	Errors / destructive
`orangecolor`	`#FF7D53`	Alerts / warm accents
`lightgrey`	`#E8ECF4`	Borders, dividers
`darkgrey`	`#DBE1FF`	Selected chip background

`containercolor`	`#DBE1FF`	Service chip backgrounds, icon containers
`backgroundColor`	`#FFFFFF`	Screen background
`backgroundColor1`	`#F7F9FB`	Section card background
`green`	`#0EBE7E`	Success states
`iconColor`	`#45464D`	Default icon color
`indicatorColor`	`#DBE1FF`	Navigation bar selected indicator

12.2 Typography

File: `text_styles.dart`

All text styles use the **Manrope** font family (`assets/fonts/`):

Style	Size	Weight	Color
`headline`	28	500 (Medium)	blackColor
`title`	24	700 (Bold)	blackColor
`title1`	20	600 (SemiBold)	blackColor
`title2`	16	500 (Medium)	titlecolor (blue)
`button`	18	500 (Medium)	titlecolor (blue)
`body1`	14	500 (Medium)	titlecolor (blue)
`body2`	14	500 (Medium)	blackColor
`caption1`	12	600 (SemiBold)	blackColor
`caption2`	12	600 (SemiBold)	iconColor (dark grey)

12.3 Theming

File: `themes.dart`

`AppThemes` provides static getters `lightTheme` and `darktheme` configuring `fontFamily`, `scaffoldBackgroundColor`, `appBarTheme`, `inputDecorationTheme`, `colorScheme`, and bottom navigation bar themes.

13. Shimmer Loading States

Files in `core/shimmer/`:

- **`ShimmerWidget`**: Base shimmer sweep animation wrapper.
- **`ListShimmer`**: Skeleton loader rows for provider lists (Home & Search).
- **`GridShimmer`**: Grid loader for photo portfolio grids.
- **`BookCardShimmer`**: Booking card placeholder loader.

14. Local Storage

File: `shared_pref.dart`

Static wrapper around `shared_preferences` storing `ktoken`, `kotp`, `kuserId`, and `kfavourite` provider list. Hive CE (`hive_ce_flutter`) is also integrated for fast offline object database caching.

15. Utility Functions

Directory: `core/functions/`

- **`navigations.dart`**: Typed context wrappers (`pushTo`, `pushReplacement`, `pushToBase`, `pop`).
- **`validation.dart`**: Form input validators (email checks, required text).

- **`custom_snake_bar.dart`**: Helper for showing styled SnackBar alerts.

16. Assets

```
assets/
├── images/
│   └── user_placeholder.png   # Fallback avatar image
├── icons/
│   ├── home.svg
│   ├── book.svg
│   ├── profile.svg
│   ├── notification.svg
│   ├── menu.svg
│   ├── filter.svg
│   └── warning.svg
├── lottie/
└── fonts/
    ├── Manrope-Bold.ttf       (weight 700)
    ├── manrope-SemiBold.ttf   (weight 600)
    └── Manrope-Regular.ttf    (weight 500)
```

17. Setup & Running the Project

Prerequisites

- Flutter SDK ≥ 3.10.4
- Dart SDK ^3.10.4
- A Firebase project with Firestore enabled
- Android Studio or VS Code with Flutter extension

Steps

1. **Clone the repository**

```
git clone <repo-url>
cd fixit-local
```

2. **Install dependencies**

```
flutter pub get
```

3. **Firebase setup**

```
dart pub global activate flutterfire_cli
flutterfire configure
```

4. **Run the app**

```
flutter run
```

5. **Generate Hive adapters** (if adding new Hive models)

```
dart run build_runner build --delete-conflicting-outputs
```

Firestore Collections

Collection	Document ID	Fields
`providers`	`provider_<n>`	`uId`, `name`, `email`, `phone`, `role`, `imageUrl`, `rating`, `specialty`, `category`, `distance`, `pricePerHour`, `isTopRated`, `about`, `servicesOffered`
`customers`	`mock_customer_id`	`uId`, `name`, `email`, `phone`, `role`, `imageUrl`, `address`

Required Firestore Indexes

- `providers` ordered by `rating` (descending) — for `getAllProviders()`
- `providers` ordered by `distance` (ascending) — for `getNearbyProviders()`

Documentation written for the FixIt Local graduation project — DEPI Program, 2026.